

Внимание и блок трансформера: собираем целиком

Multi-head attention, LayerNorm, FFN и residual connection

Источники: С. Рашка, «Создаём большую языковую модель с нуля», гл. 3.6 и 4; Vaswani et al., arxiv.org/abs/1706.03762; Stanford CS336, лекции 2 и 3

Александр Чурсин
Руководитель группы
разработки в ГК «Норильский
никель»



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

-  если меня видно и слышно
-  если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне

Правила участия

- 1 Приготовьте блокнот и ручку, чтобы записывать важные мысли и идеи
- 2 Продолжительность вебинара — 1 час 20 минут
- 3 Вы можете писать свои вопросы в чате
- 4 Запись вебинара будет доступна в личном кабинете Нетологии
- 5 Оффтоп обсуждения можно продолжить в учебном чате в ВК

Александр Чурсин

О спикере:

- TeamLead NLP в Норникеле
- Стажёр → тимлид команды из 10 за 5 лет
- Бакалавриат «Лингвистика»
- Магистр «Инженерия машинного обучения»
- Мультиагентные системы и дообучение
- Домены: мебель, туризм, медиа, металлургия



Одна голова только усредняет

Выход внимания — взвешенная сумма строк. Новых признаков усреднение не строит: нужен блок целиком.

Батч прошлого занятия: $2 \times 6 \times 3$

```
1 import torch
2 import torch.nn as nn
3
4 inputs = torch.tensor(
5     [[0.43, 0.15, 0.89], # Your   (x^1)
6      [0.55, 0.87, 0.66], # journey (x^2)
7      [0.57, 0.85, 0.64], # starts  (x^3)
8      [0.22, 0.58, 0.33], # with   (x^4)
9      [0.77, 0.25, 0.10], # one    (x^5)
10     [0.05, 0.80, 0.55]] # step   (x^6)
11 )
12 batch = torch.stack((inputs, inputs), dim=0)
```

Конструктор одной головы

```
1  # Класс прошлого занятия — переносим без изменений.  
2  class CausalAttention(nn.Module):  
3      def __init__(self, d_in, d_out, context_length, dropout, qkv_bias=False):  
4          super().__init__()  
5          self.d_out = d_out  
6          self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)  
7          self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)  
8          self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)  
9          self.dropout = nn.Dropout(dropout)  
10         self.register_buffer(  
11             'mask',  
12             torch.triu(torch.ones(context_length, context_length), diagonal=1)  
13         )
```

Прямой проход одной головы

```
1 def forward(self, x):
2     b, num_tokens, d_in = x.shape
3     keys = self.W_key(x)
4     queries = self.W_query(x)
5     values = self.W_value(x)
6     attn_scores = queries @ keys.transpose(1, 2)
7     attn_scores.masked_fill_(
8         self.mask.bool()[:num_tokens, :num_tokens], -torch.inf)
9     attn_weights = torch.softmax(
10         attn_scores / keys.shape[-1]**0.5, dim=-1)
11     attn_weights = self.dropout(attn_weights)
12     return attn_weights @ values
```

Прогон одной головы: выход $2 \times 6 \times 2$

```
1 torch.manual_seed(123)
2 context_length = batch.shape[1]
3 ca = CausalAttention(d_in=3, d_out=2, context_length=context_length, dropout=0.0)
4 context_vecs = ca(batch)
5 print("context_vecs.shape:", context_vecs.shape)
6 # context_vecs.shape: torch.Size([2, 6, 2])
```

Внимание одной формулой

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

скалярные произведения query на key делятся на корень из размера головы, softmax превращает их в веса с единичной суммой, и с этими весами складываются векторы value

От входа к матрице весов

$$(2, 6, 3) \rightarrow Q, K, V : (2, 6, 2) \rightarrow QK^T : (2, 6, 6)$$

Q , K и V — проекции входа для query, key и value. Батч из двух текстов по шесть токенов и три признака сжимается до двух; QK^T даёт квадрат по числу токенов

Формы внутри одной головы

```
1 queries = ca.W_query(batch)
2 attn_scores = queries @ ca.W_key(batch).transpose(1, 2)
3 print(queries.shape, attn_scores.shape, torch.softmax(attn_scores, dim=-1).shape)
4 # torch.Size([2, 6, 2]) torch.Size([2, 6, 6]) torch.Size([2, 6, 6])
```

План занятия

- 1 **Проекции и оси**
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

Цели занятия

- Разобрать проекцию и оси матрицы X
- Собрать multi-head attention одной широкой матрицей
- Посчитать цену внимания: параметры $4d^2$ и память под веса
- Понять нормализацию по признакам внутри токена
- Разобрать полносвязный блок и расширение в четыре раза
- Собрать блок целиком и посчитать его параметры

Что мы сейчас посчитаем

$$(6, 3) \rightarrow (6, 2)$$

проекция меняет только число признаков

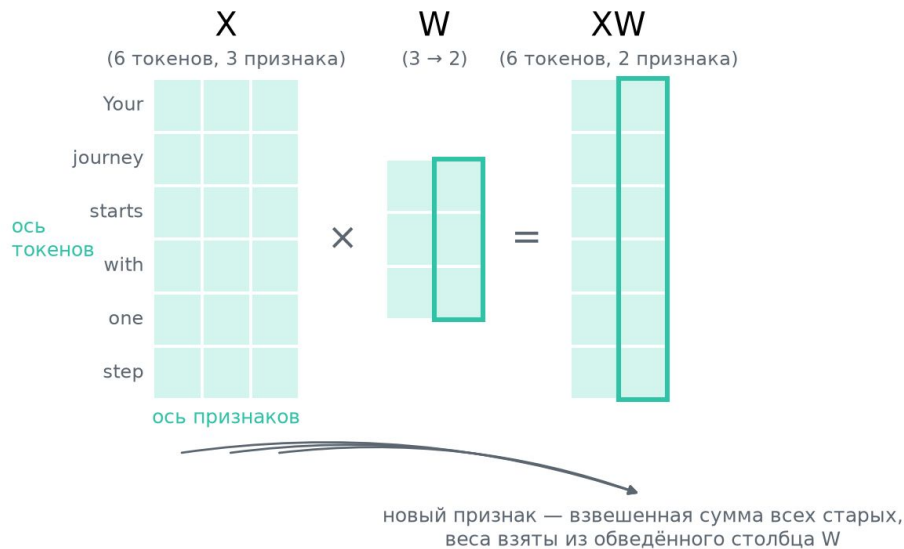
$$X \cdot W$$

новый признак — взвешенная сумма старых

Оси матрицы X: токены и признаки

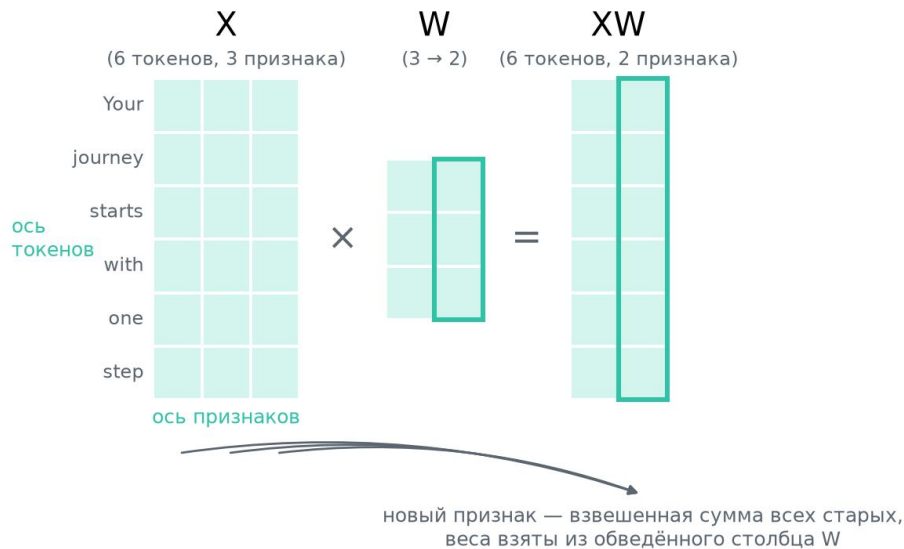
```
1 print("форма X:", inputs.shape)
2 print("ось токенов — сколько токенов:", inputs.shape[0])
3 print("ось признаков — сколько чисел на токен:", inputs.shape[1])
4 # форма X: torch.Size([6, 3])
5 # ось токенов — сколько токенов: 6
6 # ось признаков — сколько чисел на токен: 3
```

Проекция собирает новые признаки



Проекция собирает новые признаки

- Столбец XW — взвешенная сумма всех столбцов X



Проекция 3 → 2 одним слоем

```
1 torch.manual_seed(123)
2 # Слой без смещения (bias — вектор, который слой прибавляет после умножения):
3 # нам нужно чистое умножение на матрицу, как в формулах внимания.
4 projection = nn.Linear(3, 2, bias=False)
5 projected = projection(inputs)
6
7 print("было:", tuple(inputs.shape), " стало:", tuple(projected.shape))
8 # было: (6, 3) стало: (6, 2)
```

Проверка руками: взвешенная сумма

```
1 # Три веса: по одному на каждый старый признак.  
2 weights_of_first_feature = projection.weight[0]  
3 # Взвешенная сумма признаков каждого токена.  
4 by_hand = inputs @ weights_of_first_feature  
5  
6 print(by_hand)  
7 print(torch.allclose(by_hand, projected[:, 0], atol=1e-6))  
8 # tensor([-0.3536, -0.3021, -0.3015, -0.1353, -0.2052, -0.1542])  
9 # True
```

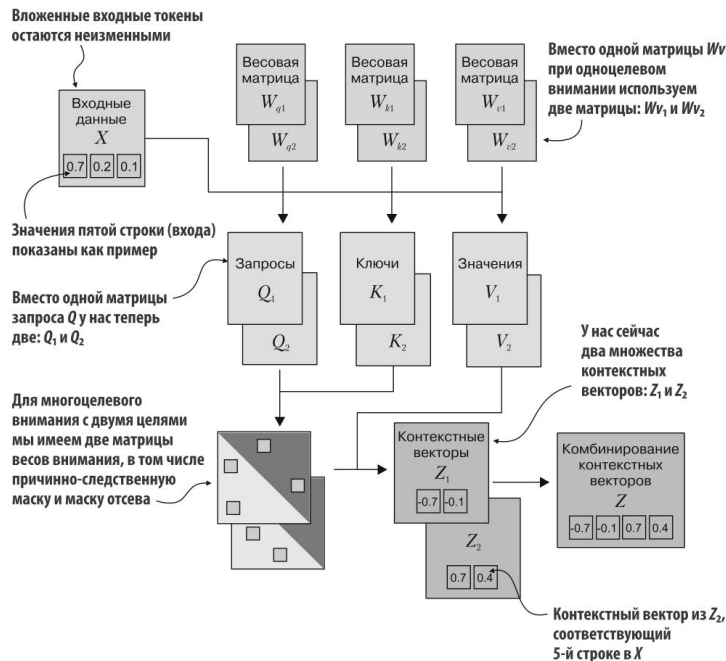
Батч меняет только левую ось

```
1 print("один текст: ", tuple(inputs.shape), "->", tuple(projection(inputs).shape))
2 print("батч из двух:", tuple(batch.shape), "->", tuple(projection(batch).shape))
3 # один текст: (6, 3) -> (6, 2)
4 # батч из двух: (2, 6, 3) -> (2, 6, 2)
```

План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной**
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

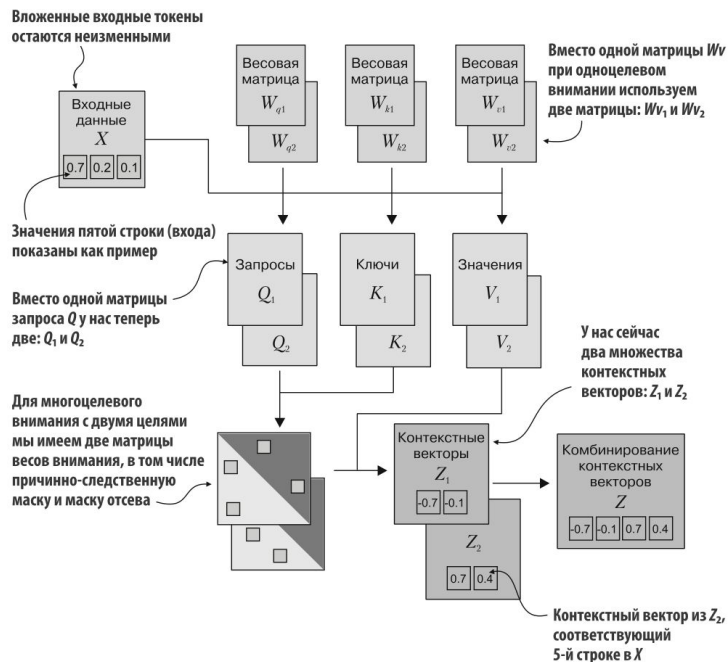
Две головы — два набора матриц



$$2 \text{ головы} \times 2 \text{ признака} = 4$$

две головы дают по два числа и вместе возвращают четыре признака

Две головы — два набора матриц

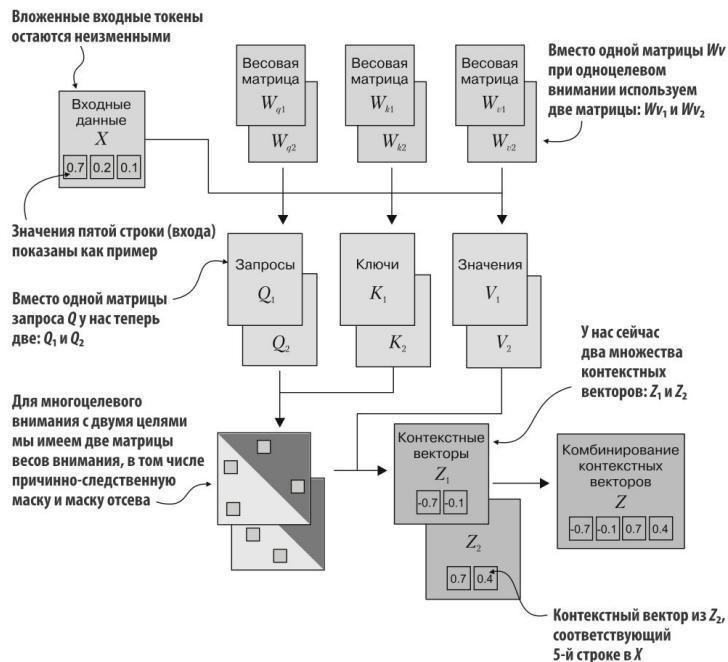


• У каждой головы свои W_q , W_k , W_v

$$2 \text{ головы} \times 2 \text{ признака} = 4$$

две головы дают по два числа и вместе возвращают четыре признака

Две головы — два набора матриц



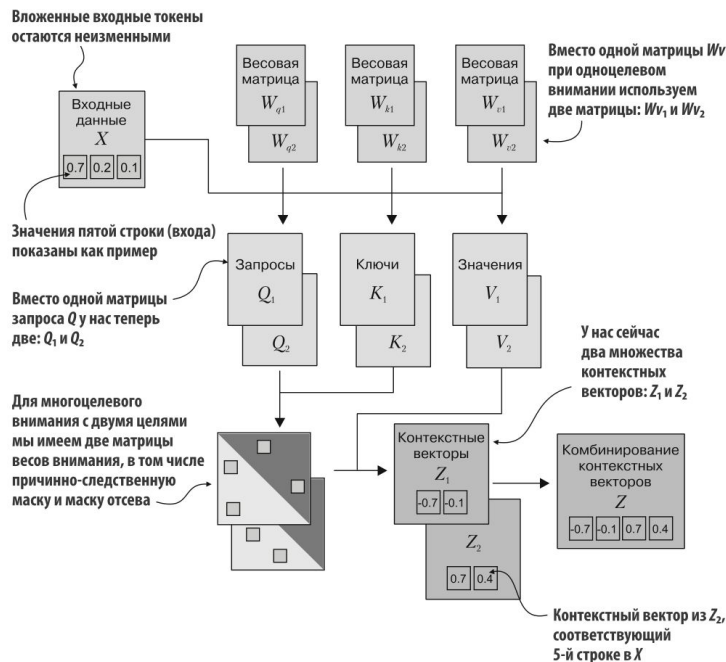
- У каждой головы свои W_q , W_k , W_v

- Своя матрица весов и своя causal mask

$$2 \text{ головы} \times 2 \text{ признака} = 4$$

две головы дают по два числа и вместе возвращают четыре признака

Две головы — два набора матриц



- У каждой головы свои W_q , W_k , W_v

- Своя матрица весов и своя causal mask

- Выходы Z_1 и Z_2 приписываются друг к другу

$$2 \text{ головы} \times 2 \text{ признака} = 4$$

две головы дают по два числа и вместе возвращают четыре признака

Обёртка: список независимых голов

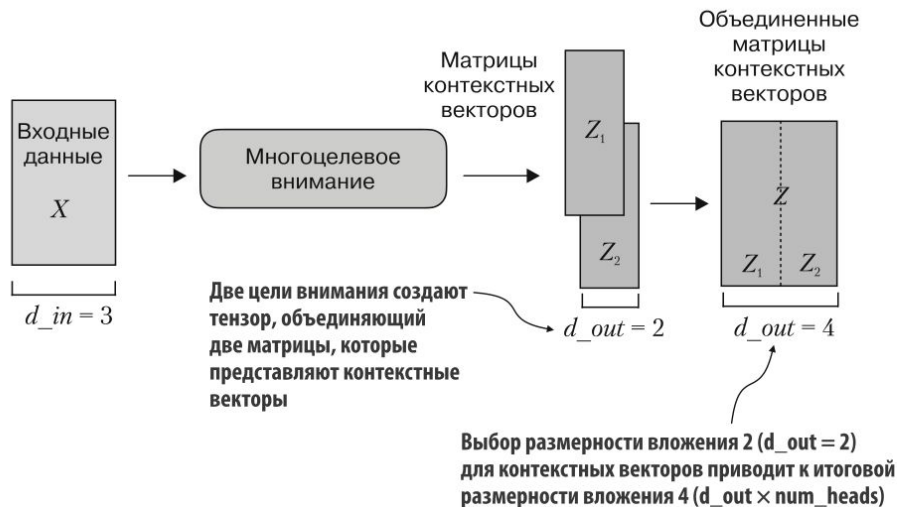
```
1 class MultiHeadAttentionWrapper(nn.Module):
2     def __init__(self, d_in, d_out, context_length,
3         dropout, num_heads, qkv_bias=False):
4         super().__init__()
5         # nn.ModuleList, а не обычный список: только так PyTorch увидит
6         # параметры каждой головы и включит их в model.parameters().
7         self.heads = nn.ModuleList(
8             [CausalAttention(
9                 d_in, d_out, context_length, dropout, qkv_bias
10             )
11             for _ in range(num_heads)]
12         )
```

Прямой проход одной головы

```
1 def forward(self, x):  
2     # Выходы голов приписываем друг к другу по последней оси —  
3     # оси признаков, поэтому dim=-1.  
4     return torch.cat([head(x) for head in self.heads], dim=-1)
```

Ширина выхода = d_{out} × число голов

- Каждая голова даёт по d_{out} чисел на токен
- Две головы по 2 числа дают 4



Почему головы — в nn.ModuleList

```
1 head_1 = CausalAttention(...)
2 head_2 = CausalAttention(...)
3 self.heads = nn.ModuleList([head_1, head_2])
```

- PyTorch регистрирует параметры всех голов
- Обычный список не передаст их оптимизатору

Две головы: выход стал шире

```
1 torch.manual_seed(123)
2 context_length = batch.shape[1] # число токенов
3 d_in, d_out = 3, 2
4 mha = MultiHeadAttentionWrapper(
5     d_in, d_out, context_length, 0.0, num_heads=2
6 )
7 context_vecs = mha(batch)
8 print(context_vecs)
9 print("context_vecs.shape:", context_vecs.shape)
10 # tensor([[[[-0.4519,  0.2216,  0.4772,  0.1063],
11 #           [-0.5874,  0.0058,  0.5891,  0.3257],
12 #           [-0.6300, -0.0632,  0.6202,  0.3860],
13 #           [-0.5675, -0.0843,  0.5478,  0.3589],
14 # context_vecs.shape: torch.Size([2, 6, 4])
```

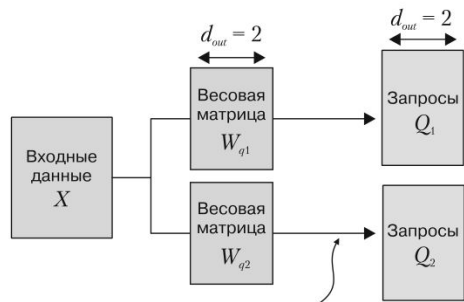
Узкие головы: снова два числа

```
1 torch.manual_seed(123)
2 mha_narrow = MultiHeadAttentionWrapper(d_in, d_out=1, context_length=context_length,
3                                         dropout=0.0, num_heads=2)
4 print("context_vecs.shape:", mha_narrow(batch).shape)
5 # context_vecs.shape: torch.Size([2, 6, 2])
```

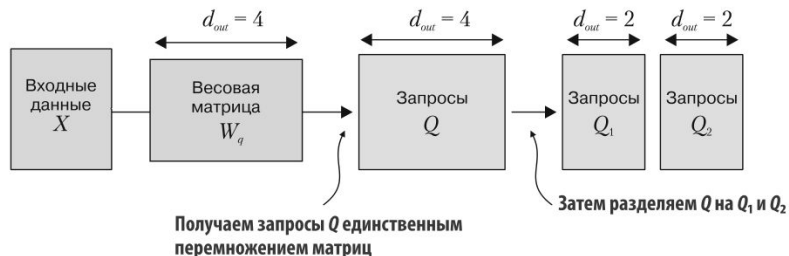
План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов**
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

Одна широкая матрица вместо двух узких



Выполняем два перемножения матриц, чтобы получить матрицы запросов Q_1 и Q_2



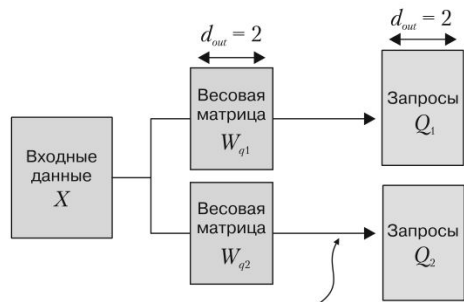
Получаем запросы Q единственным перемножением матриц

Затем разделяем Q на Q_1 и Q_2

4 признака $\longrightarrow 2 + 2$

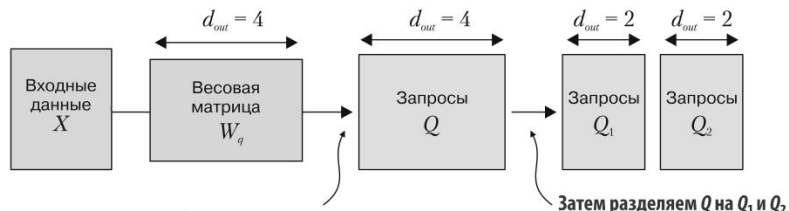
одна широкая проекция затем делится на две равные части — по одной на голову

Одна широкая матрица вместо двух узких



Выполняем два перемножения матриц, чтобы получить матрицы запросов Q_1 и Q_2

- Обёртка: два умножения на узкие W_{q1} и W_{q2}



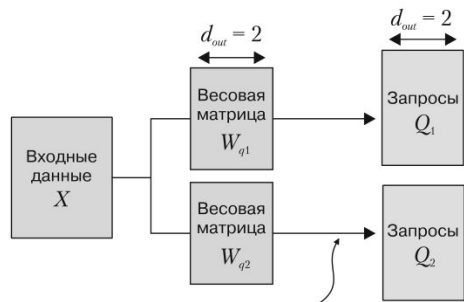
Получаем запросы Q единственным перемножением матриц

Затем разделяем Q на Q_1 и Q_2

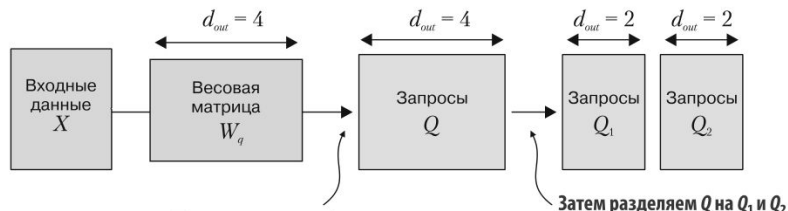
4 признака $\longrightarrow 2 + 2$

одна широкая проекция затем делится на две равные части — по одной на голову

Одна широкая матрица вместо двух узких



Выполняем два перемножения матриц, чтобы получить матрицы запросов Q_1 и Q_2



Получаем запросы Q единственным перемножением матриц

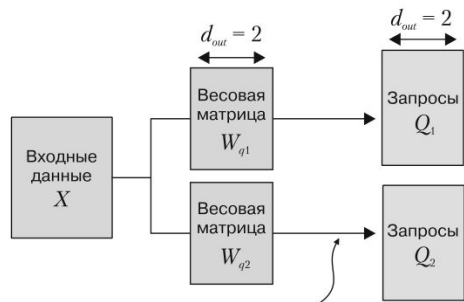
Затем разделяем Q на Q_1 и Q_2

- Обёртка: два умножения на узкие W_{q1} и W_{q2}
- Класс: одно умножение на широкую W_q

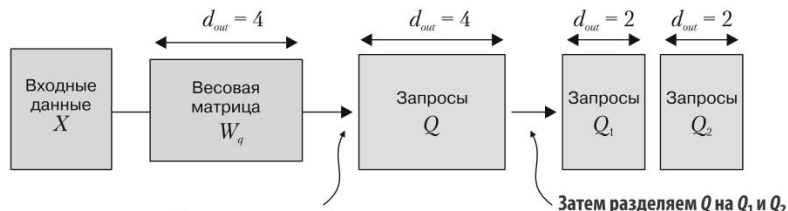
4 признака $\longrightarrow 2 + 2$

одна широкая проекция затем делится на две равные части — по одной на голову

Одна широкая матрица вместо двух узких



Выполняем два перемножения матриц, чтобы получить матрицы запросов Q_1 и Q_2



Получаем запросы Q единственным перемножением матриц

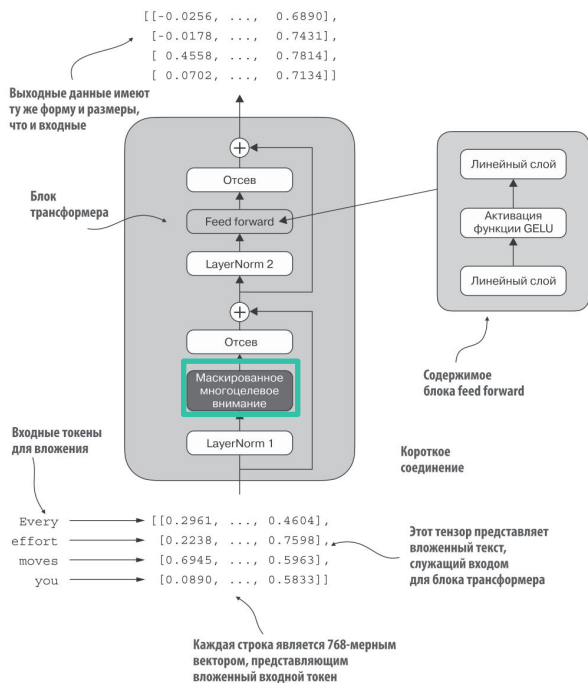
Затем разделяем Q на Q_1 и Q_2

- Обёртка: два умножения на узкие W_{q1} и W_{q2}
- Класс: одно умножение на широкую W_q
- Результат делится на Q_1 и Q_2

4 признака $\longrightarrow 2 + 2$

одна широкая проекция затем делится на две равные части — по одной на голову

Где мы в блоке



У Рашки «многоцелевое внимание» — это многоголовое.

Собираем multi-head attention — нижний подслой блока.

Формула, которую собираем в коде

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

h — число голов; W^O — выходная матрица, она сшивает головы обратно

$$\text{head}_i = \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V)$$

i — номер головы; W_i^Q, W_i^K, W_i^V — три матрицы этой головы

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \cdot V$$

Q, K, V — запросы, ключи и значения; d_k — размер одной головы

Формула, которую собираем в коде

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

h — число голов; W^O — выходная матрица, она сшивает головы обратно

$$\text{head}_i = \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V)$$

i — номер головы; W_i^Q, W_i^K, W_i^V — три матрицы этой головы

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \cdot V$$

Q, K, V — запросы, ключи и значения; d_k — размер одной головы

- Concat склеивает выходы голов: h кусков по d/h чисел дают снова d

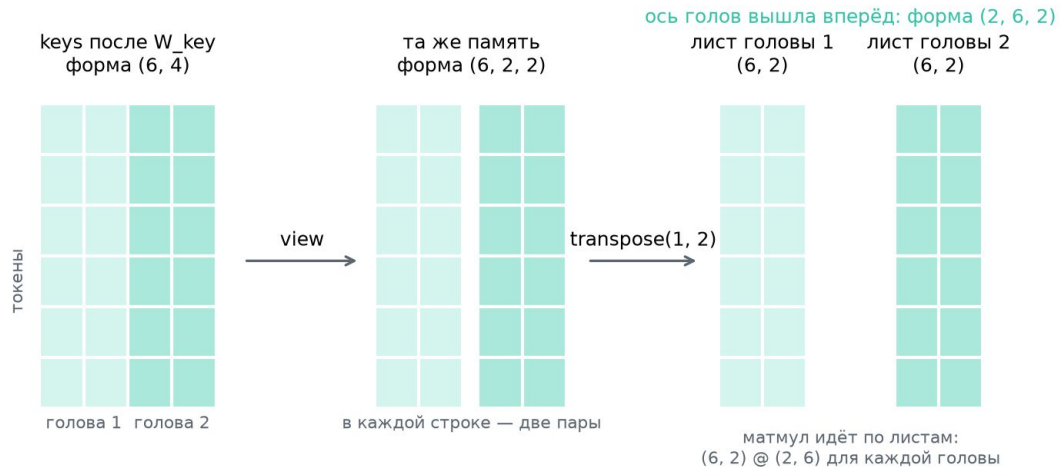
Конструктор MultiHeadAttention целиком

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_in, d_out,
3         context_length, dropout, num_heads, qkv_bias=False):
4         super().__init__()
5         assert (d_out % num_heads == 0), \
6             "d_out must be divisible by num_heads"
7
8         self.d_out = d_out
9         self.num_heads = num_heads
10        self.head_dim = d_out // num_heads # размер одной головы
11        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
12        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
13        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
14        self.out_proj = nn.Linear(d_out, d_out)
15        self.dropout = nn.Dropout(dropout)
16        self.register_buffer(
17            "mask",
18            torch.triu(torch.ones(context_length, context_length), diagonal=1)
19        )
```

Шаг 1: проекции и ось голов

```
1 def forward(self, x):
2     b, num_tokens, d_in = x.shape
3     keys = self.W_key(x)      # форма (b, num_tokens, d_out)
4     queries = self.W_query(x)
5     values = self.W_value(x)
6
7     # Неявно разделяем матрицу, добавляя ось num_heads:
8     # (b, num_tokens, d_out) -> (b, num_tokens, num_heads, head_dim)
9     keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
10    values = values.view(b, num_tokens, self.num_heads, self.head_dim)
11    queries = queries.view(
12        b, num_tokens, self.num_heads, self.head_dim
13    )
```

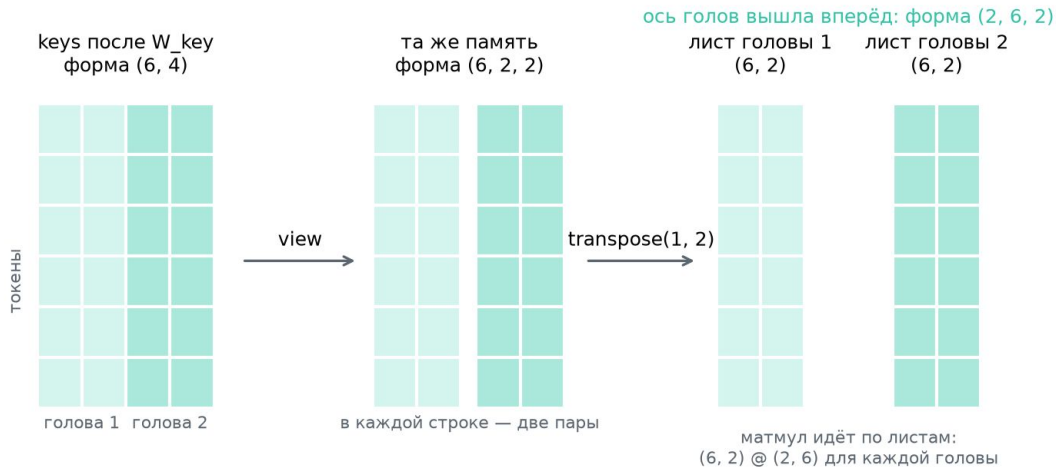
view режет, transpose выносит ось голов



цвет клетки — та же голова: данные не двигаются, меняется только группировка осей

view режет, transpose выносит ось голов

- Слева: keys (6, 4), цвет — ГОЛОВА



цвет клетки — та же голова: данные не двигаются, меняется только группировка осей

Игрушечный тензор (1, 2, 3, 4)

```
1 a = torch.tensor([[[[0.2745, 0.6584, 0.2775, 0.8573],
2                   [0.8993, 0.0390, 0.9268, 0.7388],
3                   [0.7179, 0.7058, 0.9156, 0.4340]],
4                   [[0.0772, 0.3565, 0.1479, 0.5331],
5                   [0.4066, 0.2318, 0.4545, 0.9737],
6                   [0.4606, 0.5159, 0.4220, 0.5786]]]])
7
8 # Матмул между тензором и его версией с переставленными двумя последними осями.
9 print(a @ a.transpose(2, 3))
10 # tensor([[[[1.3208, 1.1631, 1.2879],
11           #    [1.1631, 2.2150, 1.8424],
12           #    [1.2879, 1.8424, 2.0402]],
13           #    [0.5903, 1.0620, 0.9912]]]])
14
```

Две головы руками: числа разные

```
1 first_head = a[0, 0, :, :]
2 first_res = first_head @ first_head.T
3 print("First head:\n", first_res)
4 # First head:
5 # tensor([[[1.3208, 1.1631, 1.2879],
6 #          [1.1631, 2.2150, 1.8424],
7 #          [1.2879, 1.8424, 2.0402]])
8 second_head = a[0, 1, :, :]
9 second_res = second_head @ second_head.T
10 print("\nSecond head:\n", second_res)
11 # Second head:
12 # tensor([[[0.4391, 0.7003, 0.5903],
13 #          [0.7003, 1.3737, 1.0620],
14 #          [0.5903, 1.0620, 0.9912]])
```

Шаг 2: ось голов вперёд и маска

```
1  # (b, L, num_heads, head_dim) -> (b, num_heads, L, head_dim)
2  keys = keys.transpose(1, 2)
3  queries = queries.transpose(1, 2)
4  values = values.transpose(1, 2)
5
6  # Скалярные произведения для каждой головы: матмат по двум последним осям.
7  attn_scores = queries @ keys.transpose(2, 3)
8  # Маска, обрезанная по числу токенов.
9  mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
10 # Заливаем будущие позиции -inf до softmax.
11 attn_scores.masked_fill_(mask_bool, -torch.inf)
```

Шаг 3: softmax и сумма value

```
1 # keys.shape[-1] после transpose — это head_dim:
2   # делим на корень из размера головы.
3   attn_weights = torch.softmax(
4       attn_scores / keys.shape[-1]**0.5, dim=-1)
5   attn_weights = self.dropout(attn_weights)
6
7   # (b, num_heads, L, head_dim) -> (b, L, num_heads, head_dim)
8   context_vec = (attn_weights @ values).transpose(1, 2)
```

Шаг 4: склейка голов и out_proj

```
1 # Объединяем головы: self.d_out = self.num_heads * self.head_dim.  
2   context_vec = context_vec.contiguous().view(  
3       b, num_tokens, self.d_out  
4   )  
5   context_vec = self.out_proj(context_vec) # выходная проекция  
6   return context_vec
```

Формула и код: шаг за шагом

Шаг формулы	Строка кода	Форма после шага
$Q \cdot WQ, K \cdot WK, V \cdot WV$	<code>self.W_query(x)</code> и две рядом	(b, L, d_{out})
разделение на головы	<code>.view(b, num_tokens, num_heads, head_dim)</code>	(b, L, H, d_h)
голова как независимый лист	<code>.transpose(1, 2)</code>	(b, H, L, d_h)
$Q \cdot KT$	<code>queries @ keys.transpose(2, 3)</code>	(b, H, L, L)
causal mask	<code>masked_fill_(mask_bool, -torch.inf)</code>	(b, H, L, L)
$\text{softmax}(\dots / \sqrt{d_k})$	<code>torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)</code>	(b, H, L, L)

Класс целиком: как части сложились

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_in, d_out,
3         context_length, dropout, num_heads, qkv_bias=False):
4         super().__init__()
5         assert (d_out % num_heads == 0), \
6             "d_out must be divisible by num_heads"
7         self.d_out = d_out
8         self.num_heads = num_heads
9         # ... разобранные строки ...
10        context_vec = context_vec.contiguous().view(
11            b, num_tokens, self.d_out
12        )
13        context_vec = self.out_proj(context_vec) # выходная проекция
14        return context_vec
```

Почему view, а не другая форма

```
1 torch.manual_seed(123)
2 mha4 = MultiHeadAttention(d_in=3, d_out=4, context_length=context_length,
3                             dropout=0.0, num_heads=2)
4 keys = mha4.W_key(batch)
5 right = keys.view(2, 6, 2, 2).transpose(1, 2) # (b, num_heads, L, head_dim)
6 wrong = keys.view(2, 2, 6, 2) # та же форма, другое содержимое
7 print(right.shape == wrong.shape, torch.allclose(right, wrong))
8 # True False
9 # Формула руками: head_i = Attention(Q_i, K_i, V_i); Concat(head_1, head_2) @ W_O
10 q = mha4.W_query(batch).view(2, 6, 2, 2).transpose(1, 2)
11 v = mha4.W_value(batch).view(2, 6, 2, 2).transpose(1, 2)
12 scores = q @ right.transpose(2, 3)
13 scores.masked_fill_(mha4.mask.bool()[:6, :6], -torch.inf)
```

Формула руками: сходится с классом

```
1 weights = torch.softmax(scores / 2**0.5, dim=-1)    # head_dim = 2
2 heads = (weights @ v).transpose(1, 2).reshape(2, 6, 4) # конкатенация голов;
3 # reshape вместо view: он сам делает копию,
4 # поэтому contiguous() здесь не нужен
5 print(torch.allclose(mha4.out_proj(heads), mha4(batch)))
6 # True
```

Прогон класса: выход $2 \times 6 \times 2$

```
1 torch.manual_seed(123)
2 batch_size, context_length, d_in = batch.shape
3 d_out = 2
4 mha = MultiHeadAttention(d_in, d_out, context_length, 0.0, num_heads=2)
5 context_vecs = mha(batch)
6 print(context_vecs)
7 print("context_vecs.shape:", context_vecs.shape)
8 # tensor([[[[0.3190, 0.4858],
9 #          [0.2943, 0.3897],
10 #          [0.2856, 0.3593],
11 #          [0.2693, 0.3873],
12 #          [0.2639, 0.3928],
13 #          [0.2575, 0.4028]],
14 # context_vecs.shape: torch.Size([2, 6, 2])
```

Голова размера 1: что ломается

$$q \cdot k = q_1 k_1, \quad \sqrt{d_k} = \sqrt{1} = 1$$

q и k — единственные координаты $query$ и key . Их скалярное произведение — обычное произведение двух чисел, а деление на корень из размера головы ничего не меняет

- Формы сходятся, код считает, ошибки нет
- Но похожесть по одной координате не измеряется
- Знак произведения решает всё: похоже или нет

Обычный размер одной головы

$$768/12 = 1600/25 = 512/8 = 64$$

размер модели, делённый на число голов, даёт размер одной головы. Во всех трёх примерах это 64

- GPT-2 small: 12 голов, $d = 768$
- GPT-2 XL: 25 голов, $d = 1600$
- Transformer 2017: 8 голов, $d = 512$

План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания**
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

Четыре цены блока

За что платим	Формула	GPT-2 small
Параметры внимания	$4d^2$	2,36 млн на слой
Память под веса внимания	L^2 на голову	24 МБ на слой

Параметры внимания: $4d^2$ плюс bias

$$4d^2 + d = 4 \cdot 768^2 + 768 = 2\,360\,064$$

четыре матрицы $d \times d$ — query, key, value и выходная проекция — плюс вектор смещения у выходной проекции:
число голов в этот счёт не входит

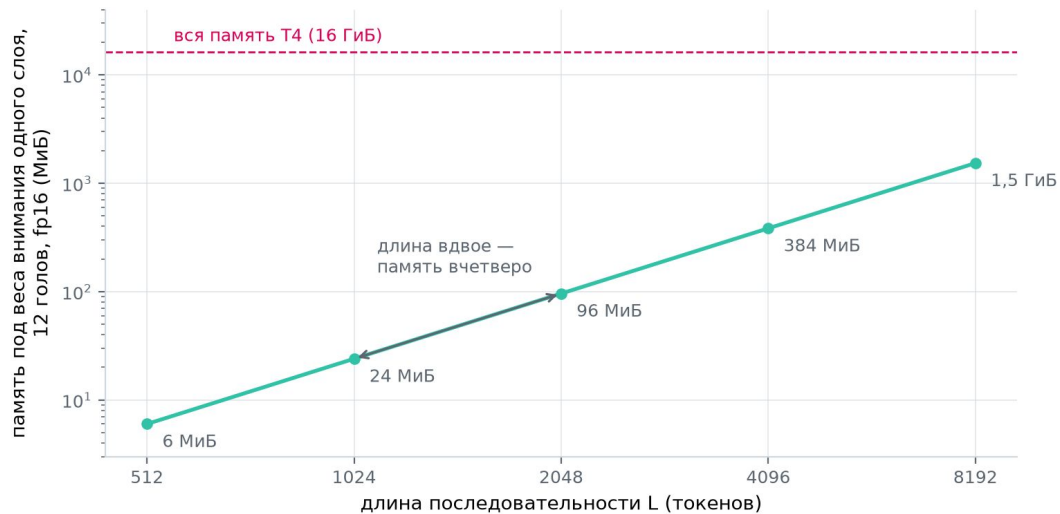
Параметры внимания: $4d^2$ плюс bias

```
1 mha_gpt2 = MultiHeadAttention(  
2     d_in=768, d_out=768, context_length=1024, dropout=0.0, num_heads=12  
3 )  
4 n_params = sum(p.numel() for p in mha_gpt2.parameters())  
5 print(f"Параметров во внимании GPT-2 small: {n_params:,}")  
6 print(f"4 * 768**2 = {4 * 768**2:,} (+ 768 в bias выходной проекции)")  
7 # Параметров во внимании GPT-2 small: 2,360,064  
8 # 4 * 768**2 = 2,359,296 (+ 768 в bias выходной проекции)
```

Память весов внимания по длинам

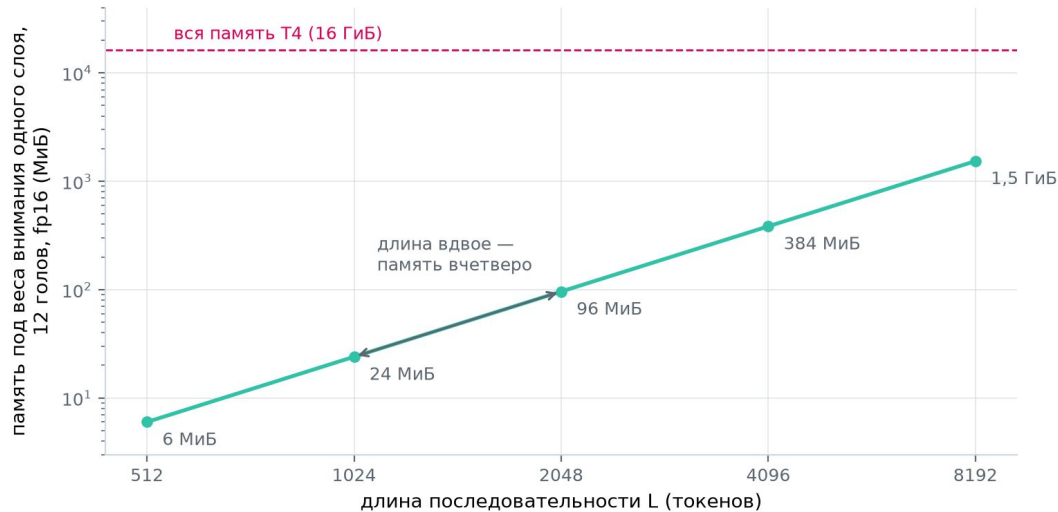
```
1 for L in (1024, 2048, 4096):
2     attn_weights = torch.zeros(1, 12, L, L, dtype=torch.float16)
3     mib = attn_weights.numel() * attn_weights.element_size() / 2**20
4     print(f"L = {L}: веса внимания одного слоя, 12 голов, fp16 — {mib:.0f} МиБ")
5 # L = 1024: веса внимания одного слоя, 12 голов, fp16 — 24 МиБ
6 # L = 2048: веса внимания одного слоя, 12 голов, fp16 — 96 МиБ
7 # L = 4096: веса внимания одного слоя, 12 голов, fp16 — 384 МиБ
```

Длина вдвое — память вчетверо



Длина вдвое — память вчетверо

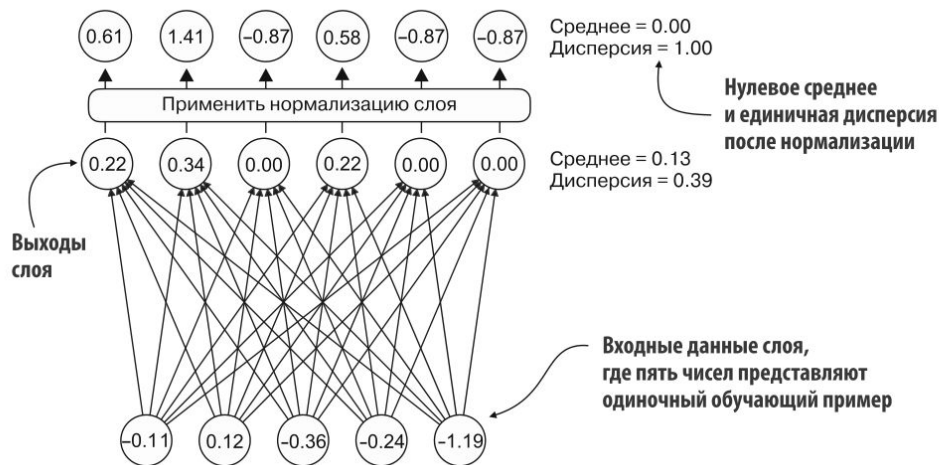
- Веса внимания одного слоя, 12 голов, fp16



План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя**
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

Нормализация приводит строку к 0 и 1

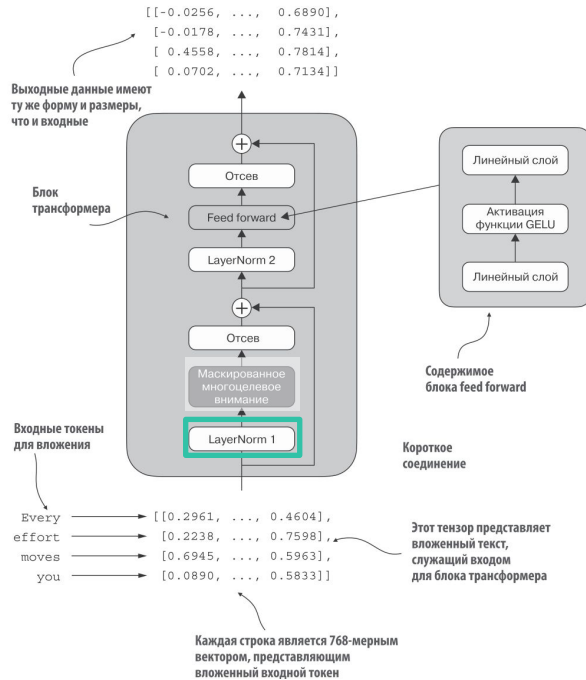


- Шесть выходов одного примера
- Среднее становится нулём, разброс — единицей

$$0,13 \longrightarrow 0, \quad 0,0231 \longrightarrow 1$$

нормализация переводит среднее строки к нулю, а её дисперсию — к единице

Где мы в блоке



Собираем нормализацию слоя:
она идёт перед каждым
подслоем.

Игрушечный пример: слой с ReLU

```
1 torch.manual_seed(123)
2 batch_example = torch.randn(2, 5) # два примера по пять признаков
3 layer = nn.Sequential(nn.Linear(5, 6), nn.ReLU())
4 out = layer(batch_example)
5 print(out)
6 # tensor([[0.2260, 0.3470, 0.0000, 0.2216, 0.0000, 0.0000],
7 #         [0.2133, 0.2394, 0.0000, 0.5198, 0.3297, 0.0000]]),
8 #       grad_fn=<ReluBackward0>)
```

Среднее и дисперсия по строке

$$\text{Var}(x) = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

x_i — i -й признак токена, μ — среднее этой строки, d — число признаков. Дисперсия измеряет средний квадрат отклонения от среднего

Среднее и дисперсия по строке

```
1 mean = out.mean(dim=-1, keepdim=True)
2 var = out.var(dim=-1, keepdim=True)
3 print("Mean:\n", mean)
4 print("Variance:\n", var)
5 # Mean:
6 # tensor([[0.1324],
7 #         [0.2170]], grad_fn=<MeanBackward1>)
8 # Variance:
9 # tensor([[0.0231],
10 #         [0.0398]], grad_fn=<VarBackward0>)
```

Ось агрегации задаёт, чьё это среднее

$\text{dim}=1$ или $\text{dim}=-1$ вычисляет среднее значение в каждом столбце, что приводит к одному среднему на строку

							Среднее
Вход 1	0.22	0.34	0.00	0.22	0.00	0.00	0.13
Вход 2	0.21	0.23	0.00	0.51	0.32	0.00	0.21

$\text{dim}=0$ вычисляет среднее значение в каждой строке, что приводит к одному среднему на столбец

Вход 1	0.22	0.34	0.00	0.22	0.00	0.00
Вход 2	0.21	0.23	0.00	0.51	0.32	0.00
Среднее	0.21	0.29	0.00	0.37	0.16	0.00

- $\text{dim}=-1$: одно число на строку, сейчас — на пример
- $\text{dim}=0$: одно число на столбец, то есть на признак
- По какой оси агрегируем, та и исчезает

Нормировка руками: 0 и 1

```
1 out_norm = (out - mean) / torch.sqrt(var)
2 mean = out_norm.mean(dim=-1, keepdim=True)
3 var = out_norm.var(dim=-1, keepdim=True)
4 print("Normalized layer outputs:\n", out_norm)
5 print("Mean:\n", mean)
6 print("Variance:\n", var)
7 # Normalized layer outputs:
8 # tensor([[ 0.6159,  1.4126, -0.8719,  0.5872, -0.8719, -0.8719],
9 #         [-0.0189,  0.1121, -1.0876,  1.5173,  0.5647, -1.0876]],
10 #        grad_fn=<DivBackward0>)
11 # Mean:
12 # tensor([[ -5.9605e-08],
13 #         [ 1.9868e-08]], grad_fn=<MeanBackward1>)
14 # [1.]], grad_fn=<VarBackward0>)
```

LayerNorm: множитель и сдвиг

```
1 class LayerNorm(nn.Module):
2     def __init__(self, emb_dim):
3         super().__init__()
4         self.eps = 1e-5
5         # nn.Parameter — тензор, который PyTorch
6         # считает обучаемым параметром модуля.
7         self.scale = nn.Parameter(torch.ones(emb_dim))
8         self.shift = nn.Parameter(torch.zeros(emb_dim))
```

Прямой проход одной головы

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} \cdot \gamma + \beta$$

x — входная строка, μ — её среднее, σ^2 — дисперсия, γ — обучаемый масштаб, β — обучаемый сдвиг, y — результат. ε не даёт делить на ноль, когда все числа строки равны

forward LayerNorm: как в GPT-2

```
1 def forward(self, x):
2     mean = x.mean(dim=-1, keepdim=True)
3     # unbiased=False: делим на n, а не на n - 1, как в GPT-2.
4     var = x.var(dim=-1, keepdim=True, unbiased=False)
5     norm_x = (x - mean) / torch.sqrt(var + self.eps)
6     return self.scale * norm_x + self.shift
7
8 ln6 = LayerNorm(emb_dim=6)
9 print(torch.allclose(ln6(out), nn.LayerNorm(6)(out), atol=1e-5))
10 # True
```

План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm**
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

Что меняет RMSNorm

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}} \cdot \gamma$$

x — строка признаков токена, d — её длина, γ (гамма) — обучаемый масштаб. RMSNorm делит на корень из среднего квадрата и не вычитает среднее

- две операции — столько пропало: вычитание среднего и сдвиг
- один проход — столько по памяти экономит RMSNorm: строку читают один раз, за средним квадратом, а не дважды

Класс RMSNorm: без среднего

```
1 class RMSNorm(nn.Module):
2     def __init__(self, emb_dim, eps=1e-5):
3         super().__init__()
4         self.eps = eps
5         self.weight = nn.Parameter(torch.ones(emb_dim)) # γ, аналог scale
6     def forward(self, x):
7         # Корень из среднего квадрата (root mean square) вдоль оси признаков.
8         rms = torch.sqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
9         return x / rms * self.weight
10 rms = RMSNorm(emb_dim=5)
11 out_rms = rms(batch_example)
12 print("RMSNorm:\n", out_rms)
13 print("корень из среднего квадрата по строкам:", out_rms.pow(2).mean(dim=-1).sqrt())
14 # tensor([1.0000, 1.0000])
```

План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок**
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

Что даёт полносвязный блок

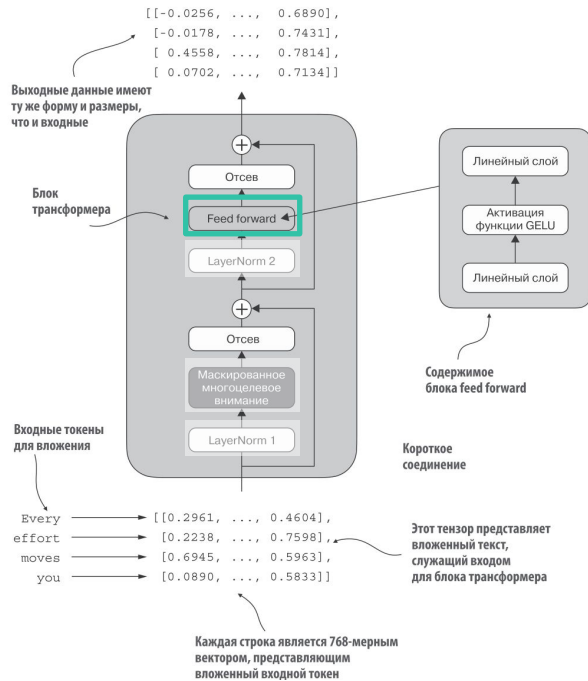
$768 \rightarrow 3072 \rightarrow 768$

расширение и сжатие вокруг
GELU

две трети

столько параметров блока
лежит в нём

Где мы в блоке



Собираем полносвязный блок
— верхний подслой.

Формула GELU: приближение через tanh

$$\text{GELU}(x) = x \cdot \Phi(x)$$

Φ — функция распределения стандартного нормального закона: вероятность, что нормальная случайная величина окажется меньше x

$$\text{GELU}(x) \approx 0,5 \cdot x \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (x + 0,044715 \cdot x^3)))$$

x — вход функции; $\tanh$ — гиперболический тангенс, он сжимает любое число в отрезок от минус единицы до единицы; π — число пи

Формула GELU: приближение через tanh

$$\text{GELU}(x) = x \cdot \Phi(x)$$

Φ — функция распределения стандартного нормального закона: вероятность, что нормальная случайная величина окажется меньше x

$$\text{GELU}(x) \approx 0,5 \cdot x \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (x + 0,044715 \cdot x^3)))$$

x — вход функции; $\tanh$ — гиперболический тангенс, он сжимает любое число в отрезок от минус единицы до единицы; π — число пи

- при больших x скобка равна 2, и $\text{GELU}(x) \approx x$ — как ReLU

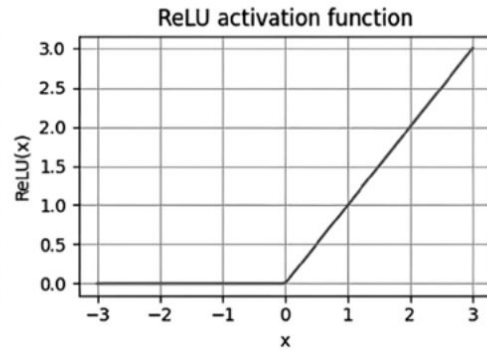
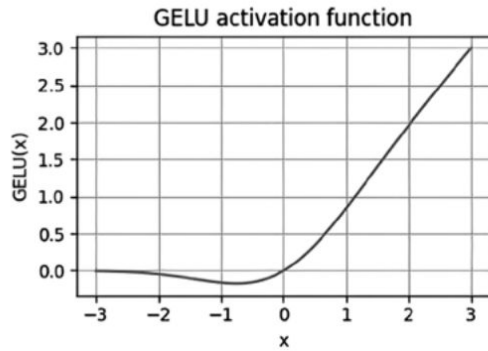
Класс GELU по формуле

```
1 class GELU(nn.Module):
2     def __init__(self):
3         super().__init__()
4
5     def forward(self, x):
6         return 0.5 * x * (1 + torch.tanh(
7             torch.sqrt(torch.tensor(2.0 / torch.pi)) *
8             (x + 0.044715 * torch.pow(x, 3))
9         ))
```

Графики GELU и ReLU рядом

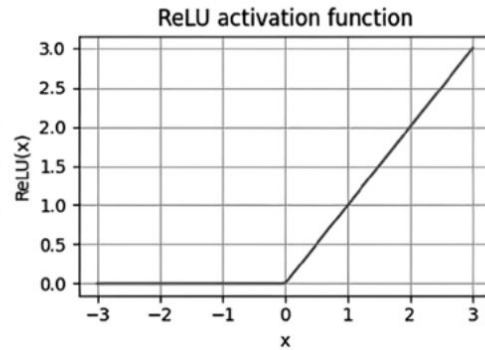
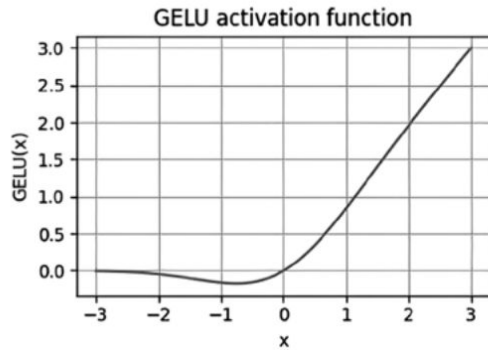
```
1 import matplotlib.pyplot as plt
2 gelu, relu = GELU(), nn.ReLU()
3 x = torch.linspace(-3, 3, 100) # сто точек от -3 до 3
4 y_gelu, y_relu = gelu(x), relu(x)
5 plt.figure(figsize=(8, 3))
6 for i, (y, label) in enumerate(zip([y_gelu, y_relu], ["GELU", "ReLU"]), 1):
7     plt.subplot(1, 2, i)
8     plt.plot(x, y)
9     plt.title(f"{label} activation function")
10    plt.xlabel("x")
11    plt.ylabel(f"{label}(x)")
12    plt.grid(True)
13 plt.tight_layout()
14 plt.show()
```

GELU гладкая, ReLU с изломом



GELU гладкая, ReLU с изломом

- ReLU: ноль слева, прямая справа



**GELU уходит
ниже нуля.
Что это значит?**

**GELU уходит
ниже нуля.
Что это значит?**

Градиент не ноль,
нейрон учится.
У ReLU был бы ноль

Конфигурация GPT-2 small

$$d = 768, \quad H = 12, \quad L = 1024$$

конфигурация GPT-2 small: d — размер вектора токена, H — число голов, L — максимальная длина контекста.
Эти значения передаются конструктору блока

Конфигурация GPT-2 small

```
1 GPT_CONFIG_124M = {
2     "vocab_size": 50257,    # размер словаря токенизатора BPE
3     "context_length": 1024, # максимальная длина контекста
4     "emb_dim": 768,        # размерность эмбединга d
5     "n_heads": 12,         # число голов внимания
6     "n_layers": 12,        # число блоков трансформера
7     "drop_rate": 0.1,      # доля dropout
8     "qkv_bias": False      # без смещений у query, key, value
9 }
10 class FeedForward(nn.Module):
```

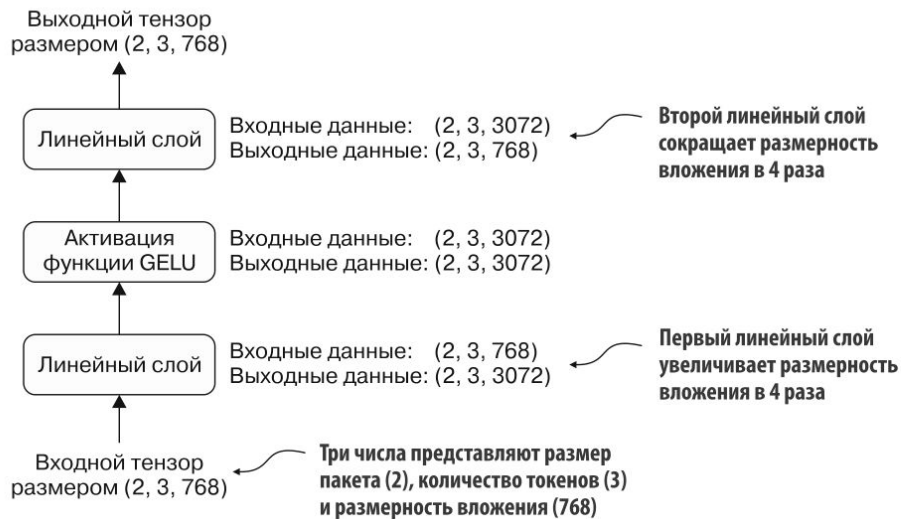
FeedForward: 768 $\rightarrow$ 3072 $\rightarrow$ 768

```
1 def __init__(self, cfg):
2     super().__init__()
3     self.layers = nn.Sequential(
4         nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]), # расширение d -> 4d
5         GELU(),
6         nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]), # сжатие 4d -> d
7     )
8 def forward(self, x):
9     return self.layers(x)
```

Форма проходит сквозь блок

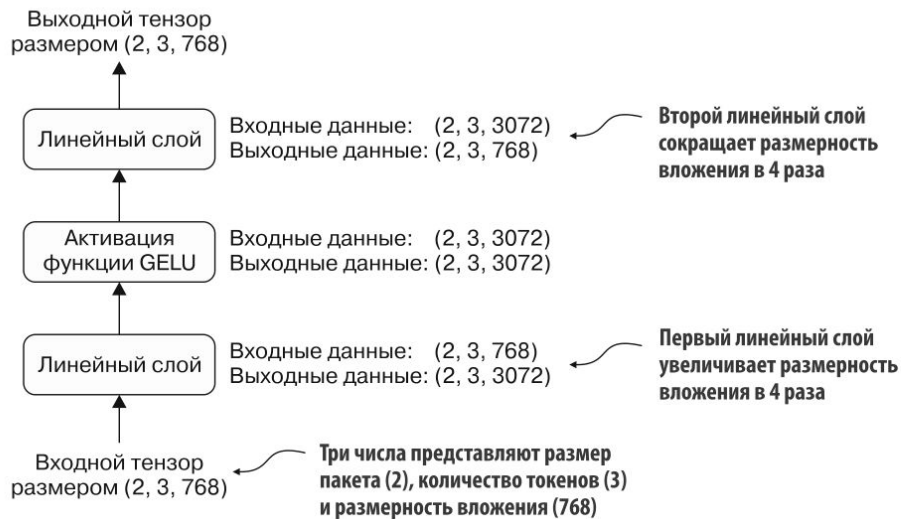
```
1 ffn = FeedForward(GPT_CONFIG_124M)
2 x = torch.rand(2, 3, 768) # батч из двух примеров по три токена
3 out = ffn(x)
4 print(out.shape)
5 # torch.Size([2, 3, 768])
```

Форма меняется только по оси признаков



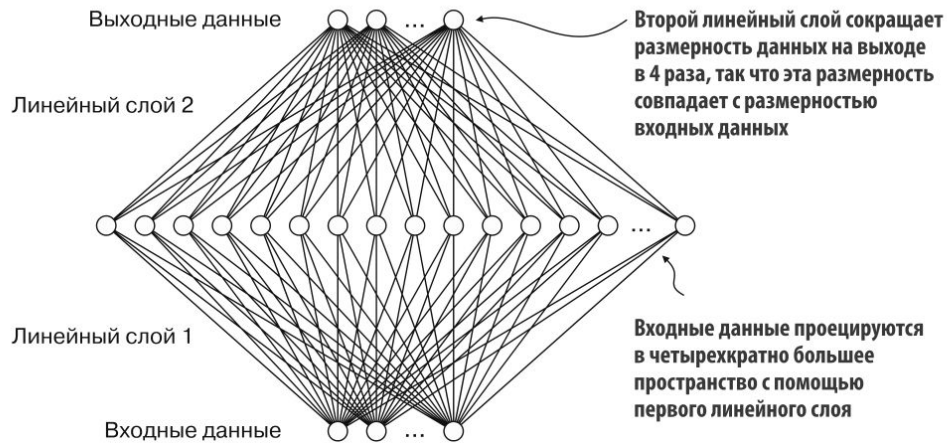
Форма меняется только по оси признаков

• $768 \rightarrow 3072 \rightarrow 3072 \rightarrow 768$



Расширить, применить GELU, сжать

- Узкий вход, широкий средний ряд, узкий выход



Параметры FFN: $8d^2$ против $4d^2$

```
1 | n_ffn = sum(p.numel() for p in ffn.parameters())
2 | print(f"Параметров в FeedForward: {n_ffn:,}")
3 | print(f"8 * 768**2 = {8 * 768**2:,} (+ {4 * 768 + 768} в смещениях)")
4 | # Параметров в FeedForward: 4,722,432
5 | # 8 * 768**2 = 4,718,592 (+ 3840 в смещениях)
```

План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями**
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер

Зачем нужна нелинейность

4,7 млн $\rightarrow$ 0,6 млн

столько параметров у двух слоёв и у одного, которым они равны без активации

ранг ≤ 768

столько независимых направлений остаётся: расширение до 3072 не добавляет ни одного

$$X \cdot W^1 \cdot W^2 = X \cdot W$$

аффинная функция от аффинной снова аффинна, сколько слоёв ни поставь подряд

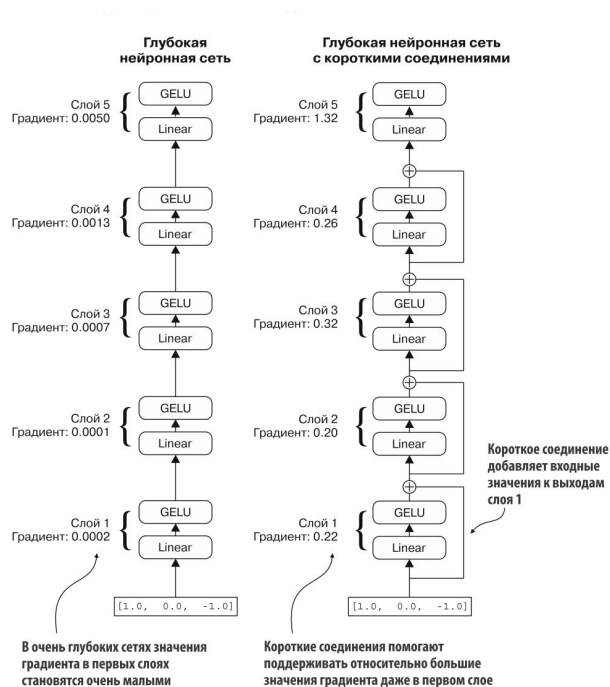
Что будет, если убрать GELU

```
1 torch.manual_seed(123)
2 x = torch.rand(2, 3, 768)
3 W1 = torch.randn(768, 3072) / 768**0.5 # масштаб как у инициализации слоя
4 W2 = torch.randn(3072, 768) / 3072**0.5
5 W = W1 @ W2 # одна матрица вместо двух
6 print("W.shape:", W.shape)
7 print(torch.allclose(x @ W1 @ W2, x @ W, atol=1e-5))
8 print("параметров в W1 и W2:", W1.numel() + W2.numel(), " в W:", W.numel())
9 # W.shape: torch.Size([768, 768])
10 # True
11 # параметров в W1 и W2: 4718592 в W: 589824
```

План занятия

- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection**
- 10 Сборка блока
- 11 Энкодер и декодер

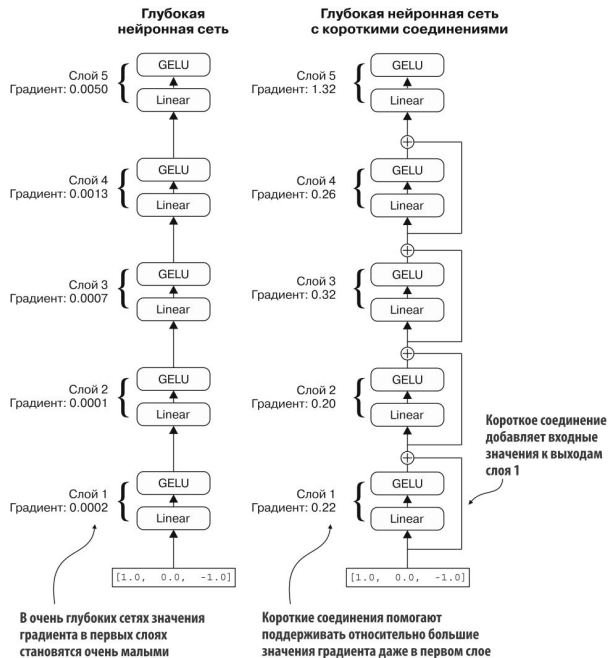
residual connection держит градиент



$$y = x + f(x), \quad \frac{\partial y}{\partial x} = I + f'(x),$$

x — вход подслоя, $f(x)$ — его преобразование. Единичная матрица I передаёт градиент напрямую, даже если производная f мала

residual connection держит градиент

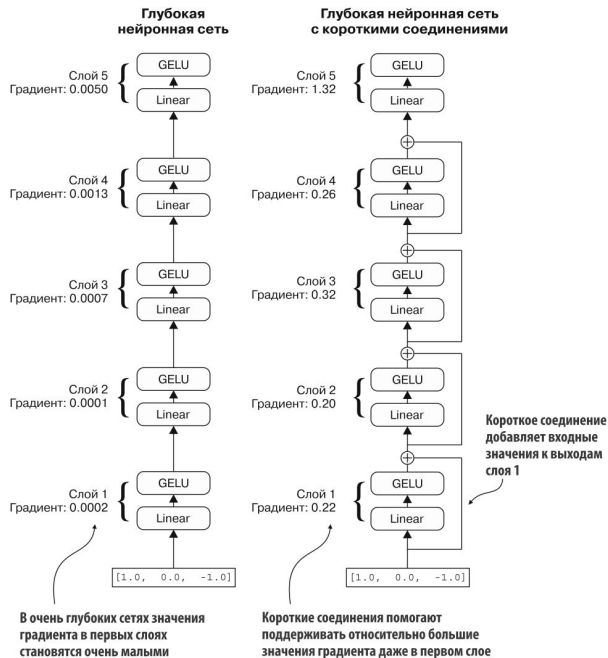


• Слева без связей: 0,0050 → 0,0002

$$y = x + f(x), \quad \frac{\partial y}{\partial x} = I + f'(x),$$

x — вход подслоя, $f(x)$ — его преобразование. Единичная матрица I передаёт градиент напрямую, даже если производная f мала

residual connection держит градиент



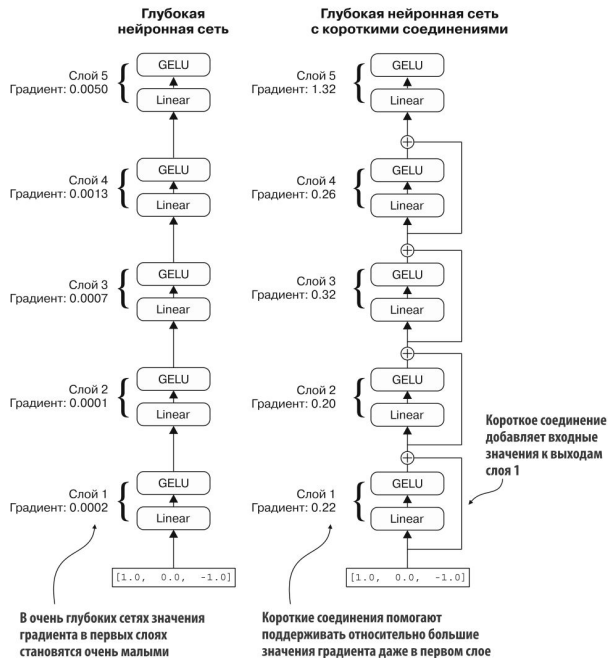
• Слева без связей: $0,0050 \rightarrow 0,0002$

• Справа со связями: $1,32 \rightarrow 0,22$

$$y = x + f(x), \quad \frac{\partial y}{\partial x} = I + f'(x),$$

x — вход подслоя, $f(x)$ — его преобразование. Единичная матрица I передаёт градиент напрямую, даже если производная f мала

residual connection держит градиент



- Слева без связей: $0,0050 \rightarrow 0,0002$

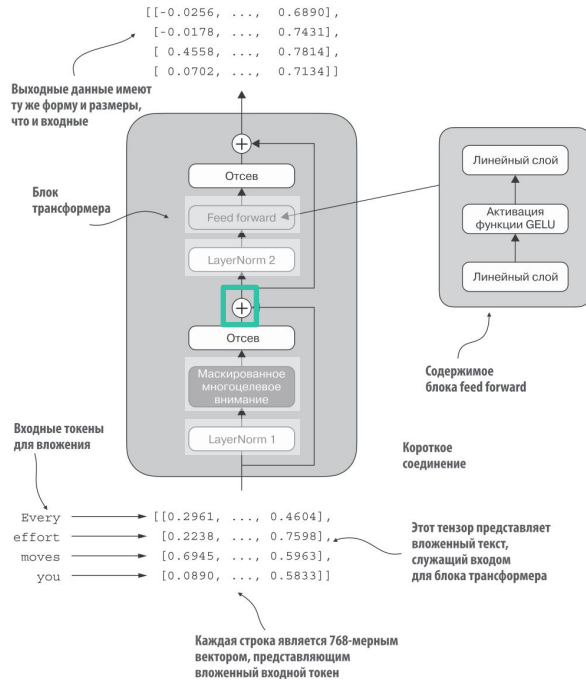
- Справа со связями: $1,32 \rightarrow 0,22$

- Дуги со знаком плюс обходят слой

$$y = x + f(x), \quad \frac{\partial y}{\partial x} = I + f'(x),$$

x — вход подслоя, $f(x)$ — его преобразование. Единичная матрица I передаёт градиент напрямую, даже если производная f мала

Где мы в блоке



Собираем residual connection:
вход подсоя складывается с
его выходом.

Пятислойная сеть с флагом связи

```
1 class ExampleDeepNeuralNetwork(nn.Module):
2     def __init__(self, layer_sizes, use_shortcut):
3         super().__init__()
4         self.use_shortcut = use_shortcut
5         self.layers = nn.ModuleList([ # пять слоёв Linear + GELU
6             nn.Sequential(nn.Linear(layer_sizes[0], layer_sizes[1]), GELU()),
7             nn.Sequential(nn.Linear(layer_sizes[1], layer_sizes[2]), GELU()),
8             nn.Sequential(nn.Linear(layer_sizes[2], layer_sizes[3]), GELU()),
9             nn.Sequential(nn.Linear(layer_sizes[3], layer_sizes[4]), GELU()),
10            nn.Sequential(nn.Linear(layer_sizes[4], layer_sizes[5]), GELU())
11        ])
```

forward: сложение со входом

$$y = x + f(x)$$

x — вход подслоя, $f(x)$ — его преобразование, y — результат. Связь не копирует веса: она только прибавляет исходный вход к поправке слоя

forward: сложение со входом

```
1 def forward(self, x):
2     for layer in self.layers:
3         layer_output = layer(x)          # выход текущего слоя
4         # Сложить можно только тензоры одной формы — отсюда проверка shape.
5         if self.use_shortcut and x.shape == layer_output.shape:
6             x = x + layer_output
7         else:
8             x = layer_output
9     return x
```

Новый пример: пять слоёв $3 \rightarrow 3$

Градиенты смотрим на игрушечной сети:
на блоке трансформера их не разглядеть.

Сеть без residual connection

```
1 layer_sizes = [3, 3, 3, 3, 3, 1]
2 sample_input = torch.tensor([[1., 0., -1.]])
3 torch.manual_seed(123) # воспроизводимая инициализация весов
4 model_without_shortcut = ExampleDeepNeuralNetwork(
5     layer_sizes, use_shortcut=False
6 )
```

Обратный проход и профиль градиентов

```
1 def print_gradients(model, x):
2     output = model(x)          # прямой проход
3     target = torch.tensor([[0.]])
4     loss = nn.MSELoss()
5     loss = loss(output, target) # потери — расстояние выхода до цели
6     loss.backward()            # обратный проход: заполняет .grad
7     for name, param in model.named_parameters():
8         if 'weight' in name:
9             # Одно число на слой: среднее абсолютное
10            # значение градиента матрицы весов.
11            print(f"{name} has gradient mean of {param.grad.abs().mean().item()}")
```

Без связей: от 0,0050 до 0,0002

```
1 print_gradients(model_without_shortcut, sample_input)
2 # layers.0.0.weight has gradient mean of 0.00020173584925942123
3 # layers.1.0.weight has gradient mean of 0.00012011158833047375
4 # layers.2.0.weight has gradient mean of 0.0007152041071094573
5 # layers.3.0.weight has gradient mean of 0.0013988735154271126
6 # layers.4.0.weight has gradient mean of 0.005049645435065031
```

Без residual connection градиент гаснет

0,0050 $\longrightarrow$ 0,0002 (25 $\times$ меньше)

на пяти слоях средний градиент первого слоя в 25 раз меньше, чем у последнего

- Каждый слой умножает градиент на свою производную
- Произведение многих чисел меньше единицы быстро стремится к нулю

С связями: градиенты не гаснут

```
1 torch.manual_seed(123)
2 model_with_shortcut = ExampleDeepNeuralNetwork(
3     layer_sizes, use_shortcut=True
4 )
5 print_gradients(model_with_shortcut, sample_input)
6 # layers.0.0.weight has gradient mean of 0.22169791162014008
7 # layers.1.0.weight has gradient mean of 0.20694106817245483
8 # layers.2.0.weight has gradient mean of 0.32896992564201355
9 # layers.3.0.weight has gradient mean of 0.2665732204914093
10 # layers.4.0.weight has gradient mean of 1.3258540630340576
```

Двенадцать слоёв: та же сеть глубже

```
1 class DeepNet(nn.Module):
2     def __init__(self, layer_sizes, use_shortcut):
3         super().__init__()
4         self.use_shortcut = use_shortcut
5         self.layers = nn.ModuleList([
6             nn.Sequential(nn.Linear(a, b), GELU())
7             for a, b in zip(layer_sizes[:-1], layer_sizes[1:])
8         ])
9     def forward(self, x):
10         for layer in self.layers:
11             out = layer(x)
12             x = x + out if self.use_shortcut and x.shape == out.shape else out
13         return x
```

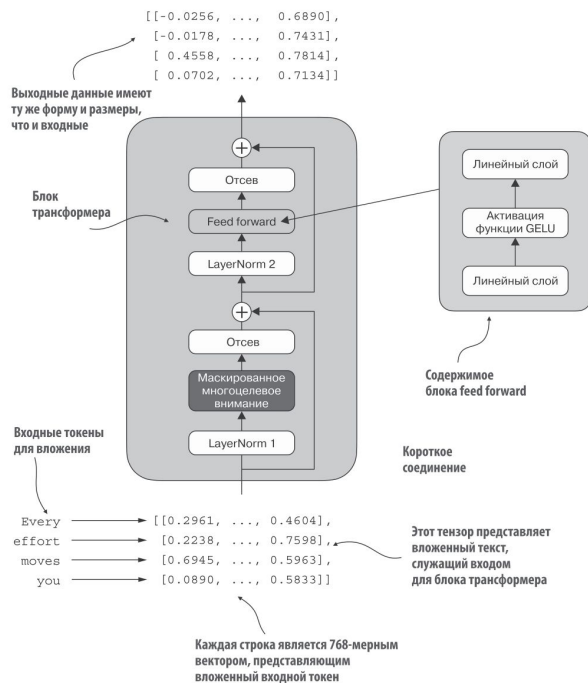
Отношение градиентов и прогон

```
1 def first_to_last(model, x):
2     loss = nn.MSELoss()(model(x), torch.tensor([[0.]])
3     loss.backward()
4     grads = [p.grad.abs().mean().item()
5               for n, p in model.named_parameters() if 'weight' in n]
6     return grads[-1] / grads[0] # отношение последнего к первому
7 torch.manual_seed(123)
8 ratio = first_to_last(DeepNet([3] * 12 + [1], False), sample_input)
9 print(f"без residual connection: {ratio:,.0f}")
10 torch.manual_seed(123)
11 ratio = first_to_last(DeepNet([3] * 12 + [1], True), sample_input)
12 print(f"с residual connection: {ratio:,.0f}")
13 # без residual connection: 1,660,108
14 # с residual connection: 8
```

План занятия

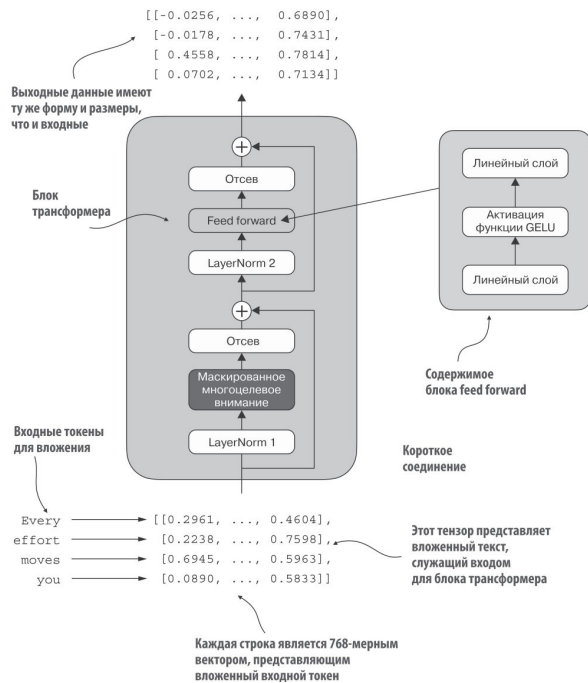
- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока**
- 11 Энкодер и декодер

Блок читается снизу вверх



embeddings $\rightarrow$ 12 $\times$ block $\rightarrow$ LayerNorm $\rightarrow$ logits
у GPT-2 small один и тот же блок
повторяется 12 раз: между эмбедингами
и выходным слоем

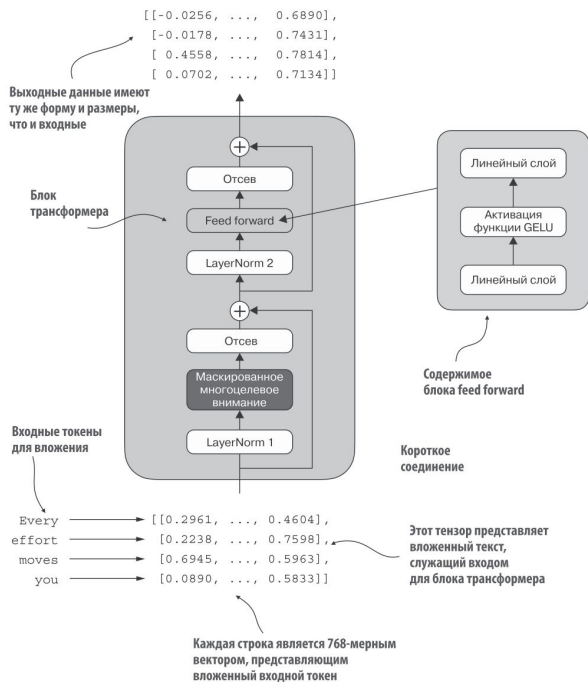
Блок читается снизу вверх



- LayerNorm, внимание, dropout, сложение

embeddings $\rightarrow$ 12 $\times$ block $\rightarrow$ LayerNorm $\rightarrow$ logits
у GPT-2 small один и тот же блок повторяется 12 раз: между эмбедингами и выходным слоем

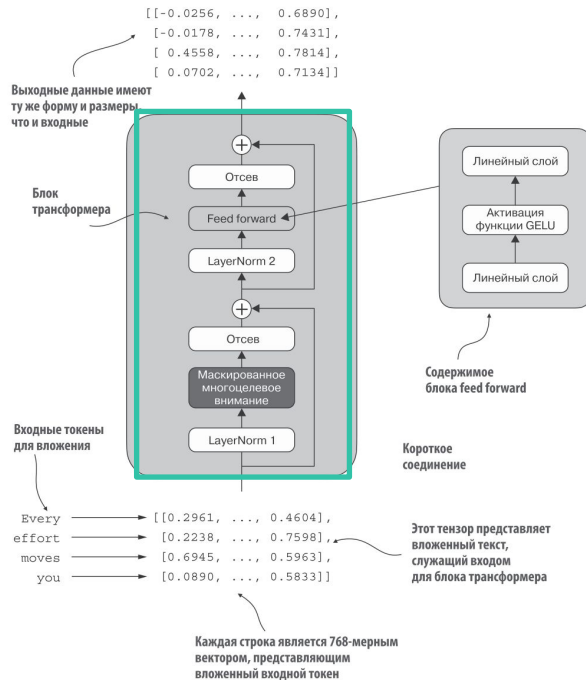
Блок читается снизу вверх



- LayerNorm, внимание, dropout, сложение
- LayerNorm, feed forward, dropout, сложение

embeddings $\rightarrow$ 12 $\times$ block $\rightarrow$ LayerNorm $\rightarrow$ logits
у GPT-2 small один и тот же блок повторяется 12 раз: между эмбедингами и выходным слоем

Где мы в блоке



Собираем блок целиком: два подслоя, у каждого норма и сложение.

Блок целиком: две одинаковые строки

$$z = x + \text{Dropout}(\text{MHA}(\text{LayerNorm}(x)))$$

x — вход блока; z — выход первого подслоя; МНА — multi-head attention

$$\text{out} = z + \text{Dropout}(\text{FFN}(\text{LayerNorm}(z)))$$

out — выход блока; FFN — полносвязный блок

- сначала нормализация, потом подслей, потом прибавление входа

Класс блока: поля конструктора

```
1 class TransformerBlock(nn.Module):
2     def __init__(self, cfg):
3         super().__init__()
4         self.att = MultiHeadAttention(
5             d_in=cfg["emb_dim"],
6             d_out=cfg["emb_dim"],
7             context_length=cfg["context_length"],
8             num_heads=cfg["n_heads"],
9             dropout=cfg["drop_rate"],
10            qkv_bias=cfg["qkv_bias"])
11        self.ff = FeedForward(cfg)
12        self.norm1 = LayerNorm(cfg["emb_dim"])
13        self.norm2 = LayerNorm(cfg["emb_dim"])
14        self.drop_shortcut = nn.Dropout(cfg["drop_rate"])
```

forward блока: две одинаковые тройки

```
1 def forward(self, x):
2     shortcut = x          # запоминаем вход для residual connection
3     x = self.norm1(x)
4     x = self.att(x)
5     x = self.drop_shortcut(x)
6     x = x + shortcut      # residual connection вокруг внимания
7
8     shortcut = x          # снова запоминаем вход
9     x = self.norm2(x)
10    x = self.ff(x)
11    x = self.drop_shortcut(x)
12    x = x + shortcut      # residual connection вокруг полносвязного блока
13    return x
```

Формула и код: шаг за шагом

Шаг формулы	Строка кода	Форма после шага
запомнить x	<code>shortcut = x</code>	(b, L, d)
$\text{LayerNorm}(x)$	<code>x = self.norm1(x)</code>	(b, L, d)
$\text{MHA}(\dots)$	<code>x = self.att(x)</code>	(b, L, d) , внутри (b, H, L, d_h)
$\text{Dropout}(\dots)$	<code>x = self.drop_shortcut(x)</code>	(b, L, d)
$x + \dots$	<code>x = x + shortcut</code>	(b, L, d) — это z
вторая строка формулы	те же пять шагов с <code>norm2</code> и <code>ff</code>	(b, L, d)

Прогон блока: форма не изменилась

```
1 torch.manual_seed(123)
2 x = torch.rand(2, 4, 768) # [batch_size, num_tokens, emb_dim]
3 block = TransformerBlock(GPT_CONFIG_124M)
4 output = block(x)
5
6 print("Input shape:", x.shape)
7 print("Output shape:", output.shape)
8 # Input shape: torch.Size([2, 4, 768])
9 # Output shape: torch.Size([2, 4, 768])
```

От одного блока к GPT-2 small

$$12 \cdot 7,09 \text{ млн} \approx 85 \text{ млн}, \quad 85 + 39 = 124 \text{ млн}$$

двенадцать одинаковых блоков дают 85 млн параметров; остальные 39 млн находятся главным образом в таблице токеновых эмбеддингов

- Внутри FFN: (2, 4, 3072); внутри внимания: (2, 12, 4, 64)
- Снаружи блок сохраняет форму (2, 4, 768)
- Одинаковая внешняя форма делает residual connection возможной

Параметры блока: 7m

```
1 def count(module):
2     return sum(p.numel() for p in module.parameters())
3
4 print(f"внимание:      {count(block.att):>9,}")
5 print(f"полносвязный блок: {count(block.ff):>9,}")
6 print(f"две нормализации: {count(block.norm1) + count(block.norm2):>9,}")
7 print(f"блок целиком:    {count(block):>9,}")
8 print(f"12 блоков GPT-2:  {12 * count(block):>9,}")
9 # внимание:      2,360,064
10 # полносвязный блок:  4,722,432
11 # две нормализации:   3,072
12 # блок целиком:      7,085,568
13 # 12 блоков GPT-2:   85,026,816
```

Post-LN: нормализация после сложения

$$\text{PostLN} : \quad y = \text{LN}(x + f(x))$$

сначала подслой $f(x)$ и residual connection, затем LayerNorm. В Pre-LN нормализация стоит перед подслоем

Post-LN: нормализация после сложения

```
1 class PostLNBlock(TransformerBlock):
2     def forward(self, x):
3         # Post-LN: сначала подслой и сложение, нормализация последней.
4         x = x + self.drop_shortcut(self.att(x))
5         x = self.norm1(x)
6         x = x + self.drop_shortcut(self.ff(x))
7         x = self.norm2(x)
8         return x
9
10 torch.manual_seed(123)
11 pre_In = TransformerBlock(GPT_CONFIG_124M).eval()
12 torch.manual_seed(123)
13 post_In = PostLNBlock(GPT_CONFIG_124M).eval()
14 # eval() включает dropout, чтобы два прогона были сравнимы.
```

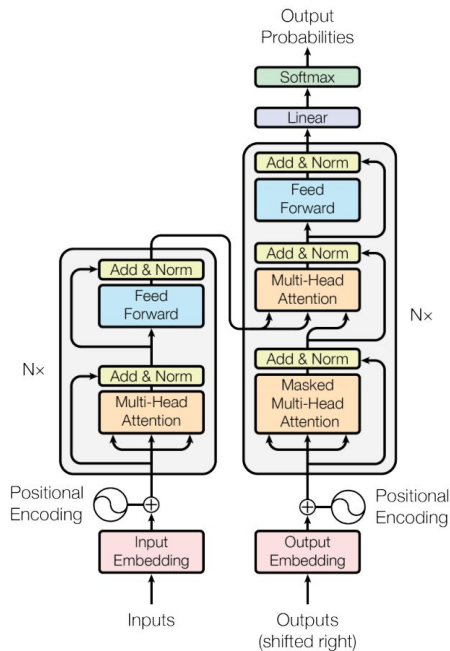
Разброс выхода и проверка в eval

```
1 print("формы совпали:", pre_ln(x).shape == post_ln(x).shape)
2 print("разброс выхода Pre-LN:", round(pre_ln(x).std().item(), 3))
3 print("разброс выхода Post-LN:", round(post_ln(x).std().item(), 3))
4 # формы совпали: True
5 # разброс выхода Pre-LN: 0.429
6 # разброс выхода Post-LN: 1.0
7
8 block.eval()
9 print(torch.allclose(block(x), block(x)))
10 block.train()
11 print(torch.allclose(block(x), block(x)))
12 # True
13 # False
```

План занятия

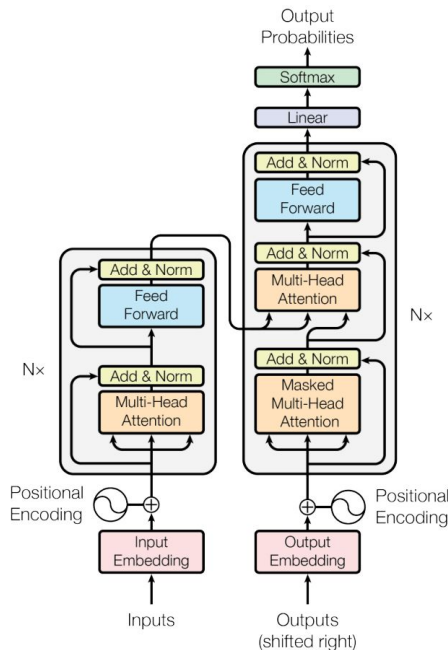
- 1 Проекции и оси
- 2 Несколько голов вместо одной
- 3 Multi-head attention с разделением весов
- 4 Цена внимания
- 5 Нормализация слоя
- 6 RMSNorm
- 7 Полносвязный блок
- 8 Нелинейность между слоями
- 9 Residual connection
- 10 Сборка блока
- 11 Энкодер и декодер**

Энкодер и декодер из тех же подслоёв

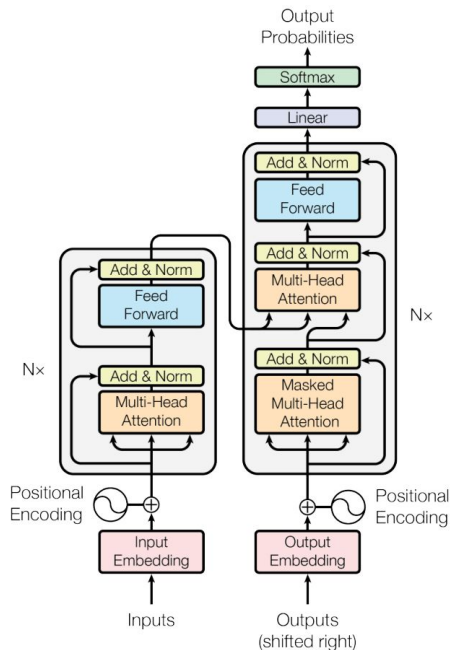


Энкодер и декодер из тех же подслоёв

- Слева энкодер: внимание и feed forward

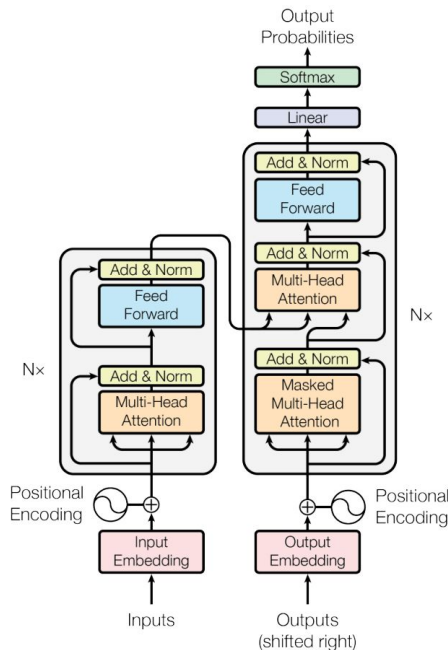


Энкодер и декодер из тех же подслоёв



- Слева энкодер: внимание и feed forward
- Справа декодер: causal mask; $N = 6$

Энкодер и декодер из тех же подслоёв



- Слева энкодер: внимание и feed forward
- Справа декодер: causal mask; $N = 6$
- Второе внимание читает выход энкодера

Энкодер и декодер: один флаг

```
1 import torch.nn.functional as F
2 torch.manual_seed(123)
3 # Одна голова, шесть токенов, размер головы 4.
4 q, k, v = torch.rand(3, 1, 1, 6, 4)
5 encoder_out = F.scaled_dot_product_attention(q, k, v, is_causal=False)
6 decoder_out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
7 last_same = torch.allclose(encoder_out[..., -1, :], decoder_out[..., -1, :])
8 print("последний токен совпал:", last_same)
9 first_same = torch.allclose(encoder_out[..., 0, :], decoder_out[..., 0, :])
10 print("первый токен совпал: ", first_same)
11 # последний токен совпал: True
12 # первый токен совпал: False
```

Блок собран и посчитан

Внимание, нормализация, полносвязный блок и residual connection — и всё это в семи миллионах параметров.

Что мы собрали

- Multi-head attention: широкая проекция и ось голов
- Цена внимания: параметры $4d^2$, память под веса как L^2
- LayerNorm — по признакам токена; RMSNorm проще
- FFN: расширение $\times 4$, GELU, две трети параметров блока
- residual connection: $I + f'(x)$ проводит градиент мимо слоя

Итоги занятия

- Читаем форму тензора и знаем, какая ось за что отвечает
- Собираем multi-head attention одной широкой матрицей
- Считаем цену слоя: $4d^2$ параметров и L^2 памяти на голову
- Отличаем нормализацию по токену от нормализации по батчу
- Объясняем, зачем блоку расширение и нелинейность
- Собираем блок целиком и считаем его параметры

Дальше: forward и backward

Профиль норм градиентов и три поломки без обучения

Дополнительные материалы

→ Vaswani et al., Attention Is All You Need: arxiv.org/abs/1706.03762

→ Shazeer, GLU Variants Improve Transformer:

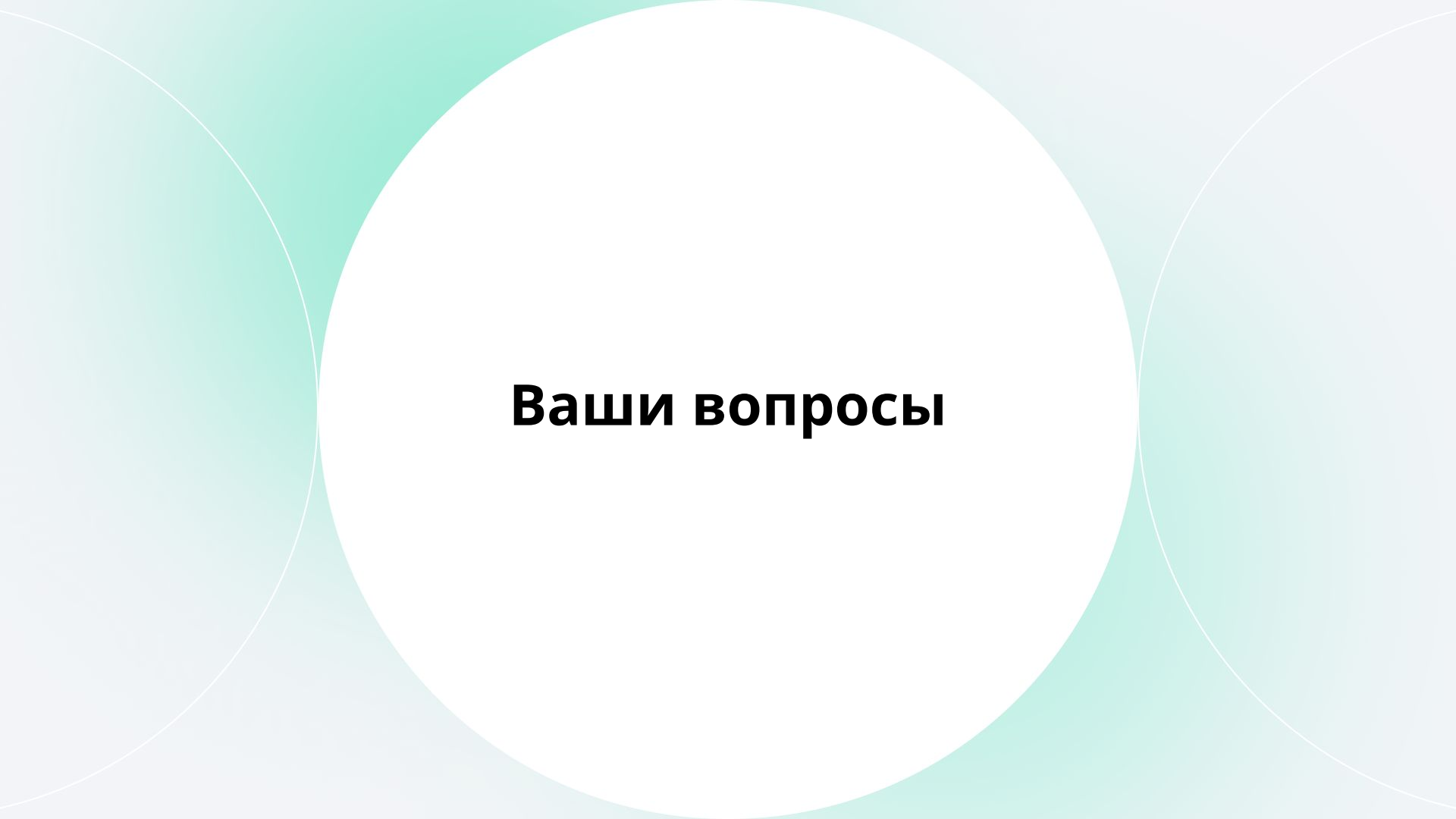
arxiv.org/abs/2002.05202

→ Xiong et al., On Layer Normalization: arxiv.org/abs/2002.04745

→ Ivanov et al., Data Movement Is All You Need:
arxiv.org/abs/2007.00072

→ Dao et al., FlashAttention: arxiv.org/abs/2205.14135





Ваши вопросы

Внимание и блок трансформера: собираем целиком

Multi-head attention, LayerNorm, FFN и residual connection

Источники: С. Рашка, «Создаём большую языковую модель с нуля», гл. 3.6 и 4; Vaswani et al., arxiv.org/abs/1706.03762; Stanford CS336, лекции 2 и 3

Александр Чурсин
Руководитель группы
разработки в ГК «Норильский
никель»

